

EXPERIMENT 30

Prolog Program for Backward Chaining

Aim

To implement **Backward Chaining** in Prolog using facts and rules, and determine whether a given goal can be proved from the available knowledge base.

Algorithm

1. Start with the required **goal**.
2. Check whether the goal is directly available as a **fact**.
3. If it is not a fact, find a **rule** whose conclusion matches the goal.
4. Treat the conditions of that rule as new sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are true, the original goal is proved.
7. If no suitable fact or rule is found, the goal fails.
8. Display the result using the required query.

Code

```
% Backward Chaining in Prolog
```

```
% Facts
```

```
has_fever(ravi).
```

```
has_cough(ravi).
```

```
has_cold(ravi).
```

```
has_fever(anu).
```

```
has_cough(anu).
```

```
% Rules
```

```
flu(X) :-  
    has_fever(X),  
    has_cough(X).
```

```
cold_flu(X) :-  
    flu(X),  
    has_cold(X).
```

```
serious_flu(X) :-  
    cold_flu(X).
```

% Required queries can be given at the Prolog prompt.

Input

```
flu(ravi).
```

Output

```
(16 ms) yes  
| ?- flu(ravi).  
  
yes  
| ?- |
```

Result

Thus, the **Backward Chaining algorithm** was successfully implemented in **Prolog**. Prolog starts from the required goal and works backwards through the rules and facts to determine whether the goal can be satisfied.